Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Estructura de Datos

Sección: A

Docente: Mario Moisés Ramírez Tobar

# “TADs y Análisis de Complejidad

# Práctica #1”

**Estudiante:**

Mynor Miguel Monzón Martínez 202230884

Quetzaltenango, 27 de agosto de

2025

# TIPOS ABSTRACTOS DE DATOS (TADs)

## TAD Nodo

Elemento fundamental para la construcción de estructuras dinámicas. Cada nodo almacena un valor y un puntero al siguiente nodo, permitiendo la creación de secuencias enlazadas.

**Dominio**:

⟨valor, siguiente⟩ donde:

- valor: Elemento de tipo genérico T
- siguiente: Puntero a otro Nodo<T> o nullptr

**Operaciones:**

**Constructor Nodo(const T& valor)**

**Prototipo**: Nodo(const T& valor)

**Precondición**: Ninguna

**Postcondición**: Crea un nodo con el valor especificado y siguiente = nullptr

**Parámetros**: valor - Elemento a almacenar

**Complejidad**: O(1)

**getValor()**

**Prototipo**: T getValor() const

**Precondición**: Ninguna

**Postcondición**: Retorna el valor almacenado sin modificar el nodo

**Retorno**: El valor almacenado

**Complejidad**: O(1)

### setValor()

**Prototipo**: void setValor(const T& nuevoValor)

**Precondición**: Ninguna

**Postcondición**: El valor del nodo se actualiza a nuevoValor

**Parámetros**: nuevoValor - Nuevo valor a almacenar

**Complejidad**: O(1)

### getSiguiente()

**Prototipo**: Nodo<T>* getSiguiente() const

**Precondición**: Ninguna

**Postcondición**: Retorna el puntero al siguiente nodo

**Retorno**: Puntero al siguiente nodo o nullptr

**Complejidad**: O(1)

### setSiguiente()

**Prototipo**: void setSiguiente(Nodo<T>* sig)

**Precondición**: Ninguna

**Postcondición**: El puntero siguiente se actualiza a sig

**Parámetros**: sig - Nuevo puntero al siguiente nodo

**Complejidad**: O(1)

## TAD LISTA ENLAZADA

Estructura lineal dinámica donde cada elemento (nodo) contiene un puntero al siguiente. En el juego se utiliza para representar la mano de cartas de cada jugador, permitiendo inserción, eliminación y búsqueda de cartas.

**Dominio**:

Secuencia ordenada de elementos de tipo T, donde:

T: Tipo genérico (en el juego: Carta*)

La lista puede estar vacía o contener n elementos ($n \geq 0$)

Cada elemento tiene una posición (1..n)

**Operaciones**:

**Constructor ListaEnlazada()**

**Prototipo**: ListaEnlazada()

**Precondición**: Ninguna

**Postcondición**: Crea una lista vacía con head = nullptr y size = 0

**Complejidad**: O(1)

**Destructor ~ListaEnlazada()**

**Prototipo**: ~ListaEnlazada()

**Precondición**: Ninguna

**Postcondición**: Libera toda la memoria de los nodos de la lista

**Complejidad**: O(n) donde n = tamaño de la lista

**isEmpty()**

**Prototipo**: bool isEmpty() const

**Precondición**: Ninguna

**Postcondición**: Retorna true si la lista está vacía, false si no

**Retorno**: true si size == 0, false en caso contrario

**Complejidad**: O(1)

### getSize()

**Prototipo**: int getSize() const

**Precondición**: Ninguna

**Postcondición**: Retorna el número de elementos en la lista

**Retorno**: Cantidad de nodos (size)

**Complejidad**: O(1)

### insertarAlFinal()

**Prototipo**: void insertarAlFinal(const T& valor)

**Precondición**: Ninguna

**Postcondición**: Agrega un nuevo nodo con 'valor' al final de la lista

**Parámetros**: valor - Elemento a insertar

**Complejidad**: O(n) en el peor caso (recorrer hasta el final)

### insertarAlInicio()

**Prototipo**: void insertarAlInicio(const T& valor)

**Precondición**: Ninguna

**Postcondición**: Agrega un nuevo nodo con 'valor' al inicio de la lista

**Parámetros**: valor - Elemento a insertar

Complejidad: O(1)

### deleteFirst()

**Prototipo**: void deleteFirst()

**Precondición**: La lista no debe estar vacía

**Postcondición**: Elimina el primer nodo de la lista y libera su memoria

**Complejidad**: O(1)

### deleteAt()

**Prototipo**: void deleteAt(int position)

**Precondición**: 1 ≤ position ≤ size

**Postcondición**: Elimina el nodo en la posición especificada

**Parámetros**: position - Posición del nodo a eliminar (1-based)

**Complejidad**: O(n) en el peor caso (recorrer hasta la posición)

### getAt()

**Prototipo**: Nodo<T>* getAt(int position) const

**Precondición**: 1 ≤ position ≤ size

**Postcondición**: Retorna el puntero al nodo en la posición especificada

**Parámetros**: position - Posición del nodo deseado (1-based)

**Retorno**: Puntero al nodo en la posición indicada

**Complejidad**: O(n) en el peor caso (recorrer hasta la posición)

### buscar()

**Prototipo**: Nodo<T>* buscar(const T& valor) const

**Precondición**: Ninguna

**Postcondición**: Busca un nodo que contenga el valor especificado

**Parámetros**: valor - Valor a buscar

**Retorno**: Puntero al nodo si lo encuentra, nullptr si no

**Complejidad**: O(n) en el peor caso (recorrer toda la lista)

**TAD LISTA CIRCULAR**

Estructura lineal dinámica donde el último nodo apunta al primero, formando un círculo. En el juego se utiliza para gestionar los turnos de los jugadores, permitiendo avanzar y retroceder de forma eficiente.

**Dominio:**

Secuencia circular de elementos de tipo T, donde:

T: Tipo genérico (en el juego: Jugador*)

La lista puede estar vacía o contener n elementos ($n \geq 0$)

Existe un puntero "actual" que indica la posición corriente

**Operaciones:**

**Constructor ListaCircular()**

**Prototipo**: ListaCircular()

**Precondición**: Ninguna

**Postcondición**: Crea una lista circular vacía con head = nullptr y actual = nullptr

**Complejidad**: O(1)

**Constructor copia ListaCircular(const ListaCircular<T>& otra)**

**Prototipo**: ListaCircular(const ListaCircular<T>& otra)

**Precondición**: Ninguna

**Postcondición**: Crea una copia profunda de la lista circular "otra"

**Parámetros**: otra - Lista circular a copiar

**Complejidad**: O(n) donde n = tamaño de la lista original

### Destructor ~ListaCircular()

**Prototipo**: ~ListaCircular()

**Precondición**: Ninguna

**Postcondición**: Libera toda la memoria de los nodos de la lista

**Complejidad**: O(n) donde n = tamaño de la lista

### isEmpty()

**Prototipo**: bool isEmpty() const

**Precondición**: Ninguna

**Postcondición**: Retorna true si la lista está vacía, false si no

**Retorno**: true si head == nullptr, false en caso contrario

**Complejidad**: O(1)

### getSize()

**Prototipo**: int getSize() const

**Precondición**: Ninguna

**Postcondición**: Calcula y retorna el número de elementos en la lista

**Retorno**: Cantidad de nodos

**Complejidad**: O(n) (debe recorrer la lista para contar)

### insertFirst()

**Prototipo**: void insertFirst(const T& value)

**Precondición**: Ninguna

**Postcondición**: Inserta un nuevo nodo al inicio de la lista circular

**Parámetros**: value - Valor a insertar

**Complejidad**: O(1) si la lista no está vacía (acceso a tail mediante getTail() que es O(n))

### insertLast()

**Prototipo**: void insertLast(const T& value)

**Precondición**: Ninguna

**Postcondición**: Inserta un nuevo nodo al final de la lista circular

**Parámetros**: value - Valor a insertar

**Complejidad**: O(1) si la lista no está vacía (acceso a tail mediante getTail() que es O(n))

### insertAt()

**Prototipo**: void insertAt(const T& value, int position)

**Precondición**: Ninguna

**Postcondición**: Inserta un nuevo nodo en la posición especificada

**Parámetros**: value - Valor a insertar, position - Posición (1-based)

**Complejidad**: O(n) en el peor caso (recorrer hasta la posición)

### getAt()

**Prototipo**: Nodo<T>* getAt(int position) const

**Precondición**: 1 ≤ position ≤ size

**Postcondición**: Retorna el puntero al nodo en la posición especificada

**Parámetros**: position - Posición del nodo deseado (1-based)

**Retorno**: Puntero al nodo en la posición indicada

**Complejidad**: O(n) (recorrer hasta la posición)

### deleteFirst()

**Prototipo**: void deleteFirst()

**Precondición**: La lista no debe estar vacía

**Postcondición**: Elimina el primer nodo de la lista y libera su memoria

**Complejidad**: O(1) si la lista no está vacía (acceso a tail mediante getTail() que es O(n))

### **deleteLast**()

**Prototipo**: void deleteLast()

**Precondición**: La lista no debe estar vacía

**Postcondición**: Elimina el último nodo de la lista y libera su memoria

**Complejidad**: O(n) (recorrer hasta el penúltimo)

### **deleteAt()**

**Prototipo**: void deleteAt(int position)

**Precondición**: 1 ≤ position ≤ size

**Postcondición**: Elimina el nodo en la posición especificada

**Parámetros**: position - Posición del nodo a eliminar (1-based)

**Complejidad**: O(n) (recorrer hasta la posición)

### **insertar() - Para manejo de turnos**

**Prototipo**: void insertar(const T& value)

**Precondición**: Ninguna

**Postcondición**: Inserta al final y si actual es nullptr, lo inicializa

**Parámetros**: value - Valor a insertar

**Complejidad**: O(1) (llama a insertLast)

### **getActual()**

**Prototipo**: Nodo<T>* getActual() const

**Precondición**: Ninguna

**Postcondición**: Retorna el puntero al nodo actual

**Retorno**: Puntero al nodo actual

**Complejidad**: O(1)

**avanzar()**

**Prototipo**: void avanzar()

**Precondición**: Lista no vacía y actual no es nullptr

**Postcondición**: actual = actual->getSiguiente()

**Complejidad**: O(1)

**retroceder()**

**Prototipo**: void retroceder()

**Precondición**: Lista no vacía

**Postcondición**: actual = nodo anterior al actual

**Complejidad**: O(n) (debe recorrer desde head hasta encontrar el anterior)

## TAD PILA

Estructura de datos lineal con acceso LIFO (Last In First Out). En el juego se utiliza para representar el mazo principal y el mazo de descarte, donde las cartas se toman desde la cima.

**Dominio**:

Secuencia de elementos de tipo T con acceso restringido al tope, donde:

T: Tipo genérico (en el juego: Carta*)

La pila puede estar vacía o contener n elementos ($n \geq 0$)

Solo se puede acceder al elemento en el tope

**Operaciones**:

**Constructor Pila()**

**Prototipo**: Pila()

**Precondición**: Ninguna

**Postcondición**: Crea una pila vacía con top = nullptr y size = 0

**Complejidad**: O(1)

**Destructor ~Pila()**

**Prototipo**: ~Pila()

**Precondición**: Ninguna

**Postcondición**: Libera toda la memoria de los nodos de la pila

**Complejidad**: O(n) donde n = tamaño de la pila

**isEmpty()**

**Prototipo**: bool isEmpty() const

**Precondición**: Ninguna

**Postcondición**: Retorna true si la pila está vacía, false si no

**Retorno**: true si top == nullptr, false en caso contrario

**Complejidad**: O(1)

**getSize()**

**Prototipo**: int getSize() const

**Precondición**: Ninguna

**Postcondición**: Retorna el número de elementos en la pila

**Retorno**: Cantidad de nodos (size)

**Complejidad**: O(1)

### push()

**Prototipo**: void push(const T& valor)

**Precondición**: Ninguna

**Postcondición**: Apila un nuevo elemento en el tope de la pila

**Parámetros**: valor - Elemento a apilar

**Complejidad**: O(1)

### pop()

**Prototipo**: T pop()

**Precondición**: La pila no debe estar vacía

**Postcondición**: Desapila y retorna el elemento del tope, liberando su nodo

**Retorno**: El valor almacenado en el nodo del tope

**Complejidad**: O(1)

### peek()

**Prototipo**: T peek() const

**Precondición**: La pila no debe estar vacía

**Postcondición**: Retorna el valor del tope sin modificar la pila

**Retorno**: El valor almacenado en el nodo del tope

**Complejidad**: O(1)

### getTop()

**Prototipo**: Nodo<T>* getTop() const

**Precondición**: Ninguna

**Postcondición**: Retorna el puntero al nodo del tope

**Retorno**: Puntero al nodo superior de la pila

**Complejidad**: O(1)

## Analisis de Complejidad

**BARAJADO (SHUFFLE)**

**Código analizado (MotorJuego.cpp):**

```
void MotorJuego::barajarMazo() {
ListaEnlazada<Carta*> temp;
while (!mazo.isEmpty()) {
temp.insertarAlFinal(mazo.pop());
}
int n = temp.getSize();
if (n < 2) return;

for (int i = 0; i < 200; i++) {
    int pos1 = (rand() % n) + 1;
    int pos2 = (rand() % n) + 1;

    if (pos1 != pos2) {
        Nodo<Carta*>* nodo1 = temp.getAt(pos1);
        Nodo<Carta*>* nodo2 = temp.getAt(pos2);

        if (nodo1 != nullptr && nodo2 != nullptr) {
```

```
            Carta* aux = nodo1->getValor();
            nodo1->setValor(nodo2->getValor());
            nodo2->setValor(aux);
        }
    }
}

for (int i = 1; i <= n; i++) {
    mazo.push(temp.getAt(i)->getValor());
}
```

**Explicación paso a paso:**

Se transfieren todas las cartas del mazo (pila) a una lista temporal.

Cada pop() es O(1)

Cada insertarAlFinal() es O(n) porque recorre hasta el final

Si hay m cartas, esta primera fase es $O(m^2)$

Se obtiene el tamaño n de la lista temporal (O(1))

Se realizan 200 iteraciones de intercambios aleatorios.

Por cada iteración, se hacen dos llamadas a getAt(), cada una O(n)

Los intercambios de valores son O(1)

Esta fase es 200 * 2 * O(n) = O(400n) = O(n)

Se transfieren las cartas de vuelta a la pila.

Por cada carta, se llama a getAt() (O(n)) y a push() (O(1))

Esta fase es n * O(n) = $O(n^2)$

**Complejidad resultante:**

La operación dominante es $O(n^2)$ debido a:

Las inserciones en la lista temporal (insertarAlFinal() es O(n) y se hace m veces)

Las lecturas con getAt() dentro de los bucles

Donde n es el número de cartas en el mazo.

**Ejemplo con valores concretos:**

Mazo de 108 cartas (1-6 jugadores): aproximadamente $108^2$ = 11,664 operaciones

Mazo de 216 cartas (7-12 jugadores): aproximadamente $216^2$ = 46,656 operaciones

**BÚSQUEDA EN MANO DEL JUGADOR**

**Contexto en el juego:**

Durante el turno de un jugador, cuando selecciona una carta para jugar, se debe acceder a esa carta en la mano y verificar si es jugable contra la carta que está en la mesa.

**Código analizado (Jugador.cpp):**

```
Carta* Jugador::verCartaEnPosicion(int posicion) {
    Nodo<Carta*>* nodo = mano.getAt(posicion);
    if (nodo != nullptr) {
        return nodo->getValor();
    }
```

```
    return nullptr;
}

Carta* Jugador::jugarCartaEnPosicion(int posicion) {
    Nodo<Carta*>* nodo = mano.getAt(posicion);
    if (nodo != nullptr) {
        Carta* c = nodo->getValor();
        mano.deleteAt(posicion);
        return c;
    }
    return nullptr;
}
```

Código relacionado (MotorJuego.cpp):

```
if (ValidadorFlip::esJugadaValida(elegida, enMesa, reglas, acumulacionActiva))
```

**Análisis de complejidad:**

Acceso a la carta por posición (getAt):

La función getAt() de ListaEnlazada recorre la lista desde head hasta la posición solicitada

En el peor caso (última posición), recorre todos los nodos

Complejidad: O(m) donde m = número de cartas en la mano

**Verificación de jugabilidad (esJugadaValida):**

Las comparaciones de colores y tipos son operaciones O(1)

No depende del tamaño de la mano

Complejidad: O(1)

**Eliminación de la carta (deleteAt):**

deleteAt() también debe recorrer la lista hasta la posición anterior a la que se eliminará

Complejidad: O(m) en el peor caso

**Casos:**

**Mejor caso: La carta elegida está en la primera posición**

getAt(1) es O(1) porque accede directamente a head

Total: O(1)

**Peor caso: La carta elegida está en la última posición**

getAt(m) recorre m nodos: O(m)

deleteAt(m) recorre hasta el nodo m-1: O(m)

Total: O(m) + O(1) + O(m) = O(m)

**Caso promedio: La carta está en una posición aleatoria**

Promedio de recorrido: m/2 nodos

Total: O(m/2) = O(m)

**Ejemplos con valores concretos:**

• Mano inicial de 7 cartas: máximo 7 operaciones para getAt()

• Mano con 10 cartas: máximo 10 operaciones

• Mano con 15 cartas (después de varias rondas): máximo 15 operaciones

**ORDENAMIENTO DE LA MANO**

**Código analizado (Jugador.cpp):**

```
int getPesoColor(Color c) {
    switch(c) {
        case AZUL: case TURQUESA: return 1;
```

```
        case ROJO: case ROSA:     return 2;
        case VERDE: case VIOLETA: return 3;
        case AMARILLO: case NARANJA: return 4;
        case NEGRO:               return 5;
        default:                  return 6;
    }
}
void Jugador::ordenarMano() {
    int n = mano.getSize();
    if (n < 2) return;

    for (int i = 1; i < n; i++) {
        for (int j = 1; j <= n - i; j++) {
            Carta* c1 = mano.getAt(j)->getValor();
            Carta* c2 = mano.getAt(j + 1)->getValor();

            bool intercambiar = false;

            int p1 = getPesoColor(c1->getColorActual());
            int p2 = getPesoColor(c2->getColorActual());

            if (p1 > p2) {
                intercambiar = true;
            }
            else if (p1 == p2) {
                if (c1->getTipoActual() > c2->getTipoActual()) {
                    intercambiar = true;
```

```
            }
        }

        if (intercambiar) {
            Nodo<Carta*>* n1 = mano.getAt(j);
            Nodo<Carta*>* n2 = mano.getAt(j + 1);
            Carta* aux = n1->getValor();
            n1->setValor(n2->getValor());
            n2->setValor(aux);
        }
    }
  }
}
```

**Análisis del algoritmo:**

**Tipo de algoritmo: Burbuja (Bubble Sort):**

Ordena primero por color (según peso definido)

Luego por tipo/valor dentro del mismo color

**Estructura de bucles:**

• Bucle externo: Se ejecuta (n-1) veces

for (int i = 1; i < n; i++)

• Bucle interno: Se ejecuta (n-i) veces en cada iteración

for (int j = 1; j <= n - i; j++)

Número total de comparaciones: Aproximadamente $n^2/2$

**Costo de cada comparación:**

**Primer par de getAt():**

mano.getAt(j) cuesta O(j) porque recorre desde el inicio

mano.getAt(j+1) cuesta O(j+1)

Total para obtener las dos cartas: O(2j+1) = O(j)

**Cálculo de pesos:**

getPesoColor() es un switch con O(1)

**Comparación de tipos:**

getTipoActual() es O(1)

Comparación de strings es O(k) donde k es longitud del string

**Segundo par de getAt() (si hay intercambio):**

Se vuelven a llamar getAt(j) y getAt(j+1) para obtener los nodos

Costo adicional O(j)

**Intercambio de valores:**

setValor() es O(1)

**Costo total:**

Para la posición j en la iteración i:

Costo ≈ O(j) para obtener las cartas

Costo ≈ O(j) para obtener los nodos (si hay intercambio)

Sumando sobre todas las iteraciones:

Aproximadamente Σ Σ j para todas las comparaciones

Esto resulta en $O(n^3)$

**Complejidad resultante:**

Complejidad temporal: $O(n^3)$ en el peor caso

Donde n = número de cartas en la mano

Factor determinante: Cada llamada a getAt() recorre la lista desde el inicio

Si se optimizara para trabajar con punteros directamente, podría reducirse a $O(n^2)$

**Ejemplos con valores concretos:**

• Mano de 7 cartas: $7^3 = 343$ operaciones aproximadas

• Mano de 10 cartas: $10^3 = 1{,}000$ operaciones aproximadas

• Mano de 15 cartas: $15^3 = 3{,}375$ operaciones aproximadas

**Justificación de aceptabilidad:**

• Las manos en UNO rara vez superan las 15-20 cartas

• 3,375 operaciones es insignificante para un computador moderno

• El ordenamiento solo se ejecuta al mostrar la mano (no en cada jugada)

## TABLA RESUMEN DE COMPLEJIDADES

| Estructura | Operación | Complejidad | Contexto en el juego |
|---|---|---|---|
| **Nodo** | Constructor | O(1) | Creación de cada carta o jugador |
| **Nodo** | Getters/Setters | O(1) | Acceso a valores y enlaces |
| **ListaEnlazada** | insertarAlInicio | O(1) | Insertar carta al principio de la mano |
| **ListaEnlazada** | insertarAlFinal | O(n) | Agregar carta robada a la mano |
| **ListaEnlazada** | deleteAt | O(n) | Eliminar carta jugada de la mano |
| **ListaEnlazada** | getAt | O(n) | Acceder a una carta en posición específica |
| **ListaEnlazada** | buscar | O(n) | Buscar una carta |

| | | | |
|---|---|---|---|
| | | | específica en la mano |
| **ListaCircular** | insertar | O(n) | Agregar nuevo jugador (por getTail) |
| **ListaCircular** | avanzar | O(1) | Pasar al siguiente turno |
| **ListaCircular** | retroceder | O(n) | Cambiar dirección del juego (requiere buscar anterior) |
| **ListaCircular** | getActual | O(1) | Obtener jugador del turno actual |
| **Pila** | push | O(1) | Poner carta en el mazo o descarte |
| **Pila** | pop | O(1) | Robar carta del mazo |
| **Pila** | peek | O(1) | Ver carta en la mesa sin sacarla |
| **MotorJuego** | barajarMazo | $O(n^2)$ | Mezclar cartas al inicio o al reponer el mazo |
| **Jugador** | verCartaEnPosicion | O(n) | Consultar una carta de la mano |
| **Jugador** | jugarCartaEnPosicion | O(n) | Jugar y eliminar una carta de la mano |
| **Jugador** | ordenarMano | $O(n^3)$ | Ordenar cartas por color y tipo para mostrar |